

```

11 import static org.junit.Assert.assertEquals;
12 /**
13  * @author Eric Zhou
14  *
15  */
16 public class StringReassemblyTest {
17
18     /*
19      * Tests of combination
20      */
21
22     /*
23      * testing combination with "hello" and "world", boundary case for overlap
24      */
25     @Test
26     public void combination_0() {
27         String result = StringReassembly.combination("hello", "world", 0);
28         assertEquals("helloworld", result);
29     }
30
31     /*
32      * testing combination with "abc" and "abc", boundary case for overlap
33      */
34     @Test
35     public void combination_Full() {
36         String result = StringReassembly.combination("abc", "abc", 3);
37         assertEquals("abc", result);
38     }
39
40     /*
41      * testing combination with "abcd" and "abcde", boundary case for overlap
42      */
43     @Test
44     public void combination_WholeWord() {
45         String result = StringReassembly.combination("abcd", "abcde", 4);
46         assertEquals("abcde", result);
47     }
48
49     /*
50      * testing combination with "abcd" and "bcde", routine test
51      */
52     @Test
53     public void combination_AtEnd() {
54         String result = StringReassembly.combination("abcd", "bcde", 3);
55         assertEquals("abcde", result);
56     }
57
58     /*
59      * testing combination with "abcd" and "bcde", large number test
60      */
61     @Test
62     public void combination_longWord() {
63         String result = StringReassembly.combination("abcdefgh", "ghijklm", 2);
64         assertEquals("abcdefghijklm", result);
65     }
66
67     /*
68      * Tests of addToSetAvoidingSubstrings
69      */
70
71     /*
72      * testing addToSetAvoidingSubstrings with "a", "ab" and "abcd", boundary
73      * case for set (need to delete the origin elements in the set)
74      */

```

```

75  @Test
76  public void addToSetAvoidingSubstrings_delete() {
77      Set<String> expected = new Set1L<>();
78      expected.add("abcd");
79      Set<String> actual = new Set1L<>();
80      actual.add("a");
81      actual.add("ab");
82      StringReassembly.addToSetAvoidingSubstrings(actual, "abcd");
83      assertEquals(expected, actual);
84  }
85
86  /*
87   * testing addToSetAvoidingSubstrings with "" and "abcd", boundary case for
88   * set (the origin set is empty, just add element)
89   */
90  @Test
91  public void addToSetAvoidingSubstrings_empty() {
92      Set<String> expected = new Set1L<>();
93      expected.add("abcd");
94      Set<String> actual = new Set1L<>();
95      StringReassembly.addToSetAvoidingSubstrings(actual, "abcd");
96      assertEquals(expected, actual);
97  }
98
99  /*
100   * testing addToSetAvoidingSubstrings with "xyz" and "abcd", boundary case
101   * for set (different string, just add new one)
102   */
103  @Test
104  public void addToSetAvoidingSubstrings_different() {
105      Set<String> expected = new Set1L<>();
106      expected.add("abcd");
107      expected.add("xyz");
108      Set<String> actual = new Set1L<>();
109      actual.add("xyz");
110      StringReassembly.addToSetAvoidingSubstrings(actual, "abcd");
111      assertEquals(expected, actual);
112  }
113
114  /*
115   * testing addToSetAvoidingSubstrings with "abcdef" and "abcd", boundary
116   * case for set (new string is the substring of the original one, do
117   * nothing)
118   */
119  @Test
120  public void addToSetAvoidingSubstrings_keep() {
121      Set<String> expected = new Set1L<>();
122      expected.add("abcdef");
123      Set<String> actual = new Set1L<>();
124      actual.add("abcdef");
125      StringReassembly.addToSetAvoidingSubstrings(actual, "abcd");
126      assertEquals(expected, actual);
127  }
128
129  /*
130   * Tests of printWithLineSeparators
131   */
132
133  /*
134   * testing printWithLineSeparators with 3 "~", routine test
135   */
136  @Test
137  public void printWithLineSeparators_3() {
138      String input = "Hello~World~Test";
139      final String fileName = "testOutput1.txt";

```

```

140 SimpleWriter out = new SimpleWriter1L(fileName);
141 StringReassembly.printWithLineSeparators(input, out);
142 out.close();
143
144 SimpleReader in = new SimpleReader1L(fileName);
145 assertEquals("Hello", in.nextLine());
146 assertEquals("World", in.nextLine());
147 assertEquals("Test", in.nextLine());
148 in.close();
149 }
150
151 /*
152  * testing printWithLineSeparators with no tilde, boundary test
153  */
154 @Test
155 public void testPrintWithLineSeparators_NoTilde() {
156     String input = "This is one line";
157     final String fileName = "testOutput2.txt";
158     SimpleWriter out = new SimpleWriter1L(fileName);
159     StringReassembly.printWithLineSeparators(input, out);
160     out.close();
161
162     SimpleReader in = new SimpleReader1L(fileName);
163     assertEquals("This is one line", in.nextLine());
164     in.close();
165 }
166
167 /*
168  * testing printWithLineSeparators with tilde at end, boundary test
169  */
170 @Test
171 public void testPrintWithLineSeparators_TildeAtEnd() {
172     String input = "One~Two~";
173     final String fileName = "testOutput3.txt";
174     SimpleWriter out = new SimpleWriter1L(fileName);
175     StringReassembly.printWithLineSeparators(input, out);
176     out.close();
177
178     SimpleReader in = new SimpleReader1L(fileName);
179     assertEquals("One", in.nextLine());
180     assertEquals("Two", in.nextLine());
181     in.close();
182 }
183
184 /*
185  * testing printWithLineSeparators with 7 tilde, large number test
186  */
187 @Test
188 public void testPrintWithLineSeparators_7() {
189     String input = "A~B~C~D~E~F~G~H";
190     final String fileName = "testOutput3.txt";
191     SimpleWriter out = new SimpleWriter1L(fileName);
192     StringReassembly.printWithLineSeparators(input, out);
193     out.close();
194
195     SimpleReader in = new SimpleReader1L(fileName);
196     assertEquals("A", in.nextLine());
197     assertEquals("B", in.nextLine());
198     assertEquals("C", in.nextLine());
199     assertEquals("D", in.nextLine());
200     assertEquals("E", in.nextLine());
201     assertEquals("F", in.nextLine());
202     assertEquals("G", in.nextLine());
203     assertEquals("H", in.nextLine());
204     in.close();

```

```

205     }
206
207     /*
208     * Tests of linesFromInput
209     */
210
211     /*
212     * testing linesFromInput with two substring, routine test
213     */
214     @Test
215     public void linesFromInput_debate2() {
216         final String fileName = "testInputLines.txt";
217
218         SimpleWriter writer = new SimpleWriter1L(fileName);
219         writer.println("abc");
220         writer.println("abcd");
221         writer.println("bcd");
222         writer.println("xyz");
223         writer.close();
224
225         SimpleReader reader = new SimpleReader1L(fileName);
226         Set<String> actual = StringReassembly.LinesFromInput(reader);
227         reader.close();
228
229         Set<String> expected = new Set1L<>();
230         expected.add("abcd");
231         expected.add("xyz");
232
233         assertEquals(expected, actual);
234     }
235
236     /*
237     * testing linesFromInput with no substring, boundary test
238     */
239     @Test
240     public void testLinesFromInput_NoSubstrings() {
241         final String fileName = "testInputNoSubstrings.txt";
242
243         SimpleWriter writer = new SimpleWriter1L(fileName);
244         writer.println("cat");
245         writer.println("dog");
246         writer.println("fish");
247         writer.println("bird");
248         writer.close();
249
250         SimpleReader reader = new SimpleReader1L(fileName);
251         Set<String> actual = StringReassembly.LinesFromInput(reader);
252         reader.close();
253
254         Set<String> expected = new Set1L<>();
255         expected.add("cat");
256         expected.add("dog");
257         expected.add("fish");
258         expected.add("bird");
259
260         assertEquals(expected, actual);
261     }
262
263
264     /*
265     * testing linesFromInput with one substring, routine test
266     */
267     @Test
268     public void testLinesFromInput_OnlyOneSubstringDeleted() {
269         final String fileName = "testInputOneSubstring.txt";

```

```
270
271     SimpleWriter writer = new SimpleWriter1L(fileName);
272     writer.println("hello");
273     writer.println("world");
274     writer.println("hell");
275     writer.close();
276
277     SimpleReader reader = new SimpleReader1L(fileName);
278     Set<String> actual = StringReassembly.LinesFromInput(reader);
279     reader.close();
280
281     Set<String> expected = new Set1L<>();
282     expected.add("hello");
283     expected.add("world");
284
285     assertEquals(expected, actual);
286 }
287
288 }
289
```